Androidネイティブ領域における間接呼び出し解決に基づくJNIコールバック静的解析の検出網羅性向上

佐藤 大樹[1,a)]　稲吉 弘樹[2,b)]　掛井 将平[1,c)]　齋藤 彰一[1,d)]

概要：Android のネイティブコードでは，JNI コールバックを用いることで，Java 層を対象とする静的解析から悪性処理を隠蔽できる．このため JNI コールバックの特定は Android アプリ解析に不可欠である．先行研究は引数意味解析等により呼び出し先を特定したが，評価は実アプリでの検出数の比較にとどまり，検出網羅性は検証されていなかった．本稿では，C 言語の JNI 呼び出しが関数ポインタテーブル経由の間接呼び出しにコンパイルされ，シンボル実体を持たないために既存の列挙手法では起点を得られない点を課題として特定する．そのうえで，全関数の P-Code 命令列から間接呼び出しを直接列挙し，呼び出し先アドレスの算出に用いられる定数オフセットを JNI 関数テーブルと照合して関数名を復元する手法を提案する．C 言語および C++による 163 種類の実装パターンを持つ正解データ付き検証用データセットを構築して評価した結果，再現率は C 言語単体で 0.0%から 97.0%へ，全体で 41.7%から 95.6%へ向上した．また，12,240 件の APK の実態調査により，ネイティブライブラリの約 4 割が C 言語実装であり，本手法が実運用上不可欠であることを確認した．

Improving Recall of JNI Callback Static Analysis via Indirect Call Resolution in Android Native Code

Daiki Sato[1,a)]　Hiroki Inayoshi[2,b)]　Shohei Kakei[1,c)]　Shoichi Saito[1,d)]

Abstract: In Android native code, JNI callbacks can be used to hide malicious behavior from Java-side static analysis. Therefore, identifying JNI callbacks is indispensable for Android app analysis. Prior research identified JNI callback targets using argument semantic analysis and evaluated the method by comparing detection counts in real-world apps. This paper constructs a validation app with ground truth data (comprising 163 patterns in total for C/C++) to evaluate detection accuracy and evaluates the proposed method based on recall. Since JNI calls in the C language are indirect calls via function pointer tables and lose name information, we implemented a process to directly detect indirect calls from the Ghidra P-Code of all functions, and to restore function names by matching the constant offsets used for target address calculation with type information. In our evaluation, the recall improved from 82.5% to 94.2% for C++, and from 41.7% to 95.6% for the evaluation including the C language. Notably, for C alone, the recall significantly increased from 0.0% to 97.0%. Furthermore, a survey of real-world apps revealed that approximately 60% of libraries are written in C++ and 40% in C, confirming that supporting both languages is crucial for practical operations.

1. はじめに

Android アプリの開発では，性能の向上や既存の C 言語および C++のコード資産の再利用を目的として，ネイティブ言語でロジックを実装する形態が広く普及している．一方でネイティブ領域は，Java 層を対象とする静的解

[1] 名古屋工業大学
Nagoya Institute of Technology
[2] 岡山大学
Okayama University
[a)] cnj14066@nitech.jp
[b)] inayoshi@okayama-u.ac.jp
[c)] kakei.shohei@nitech.ac.jp
[d)] shoichi@nitech.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

析の死角となるため，悪性処理や機密情報へのアクセスを隠蔽する手段としても悪用されていることが報告されている [1], [2].

ネイティブ領域から Java 層のオブジェクト生成，メソッド呼び出し，フィールドアクセスを行う際には，Java Native Interface である JNI[3], [4] が提供する `FindClass` 関数や `GetMethodID` 関数，`CallIntMethod` 関数や `CallCharMethod` 関数（以降これらを `Call*Method` 関数と総称する）などの関数群を用いた JNI コールバック機構が利用される．JNI コールバックは Java 層の静的解析ツールを迂回して悪性処理を実行する手段となり得るため，その呼び出し位置であるコールサイトおよび呼び出し先の正確な特定は Android アプリ解析に不可欠である [5]. 先行研究 [6] では，データフロー解析や引数意味解析によって JNI コールバックの呼び出し先を特定する手法が提案された．しかし，先行研究 P-Tracer[6] をはじめとする従来の JNI 静的解析ツールの性能評価は，主に実アプリで検出された呼び出し先の絶対数の比較にとどまっており，実際の呼び出し箇所のうちどの程度を抽出できているかという検出網羅性，すなわち再現率である Recall に基づく評価が行われていない．

本研究における詳細なバイナリ解析により，C 言語での JNI 呼び出し形式である `(*env)->Call*Method` は，コンパイル時に関数ポインタテーブルを経由する間接呼び出し命令である CALLIND 命令に変換され，呼び出し先に対応するシンボル実体を失うことが明らかとなった．このため，直接呼び出し命令である CALL 命令に対するクロスリファレンスである XREF を辿る従来の列挙手法では，C 言語で実装された JNI コールバックの Recall が 0.0%となる構造的な課題が存在していた．

さらに，先行研究 [6] のアプリセット（国内 Google Play および AndroZoo[7]）から抽出した 12,240 件の APK の実態調査では，ネイティブライブラリを含む 5,696 件の APK から抽出した 114,882 本のうち 38.2%が C 言語実装であった（付録 A.6）．すなわち間接呼び出しの解決は理論上の課題ではなく，実運用上不可欠な機能である．

本稿の貢献は以下の 3 点である．

(1) 既存ツールが列挙をシンボル XREF に依存していたために，C 言語の JNI 呼び出しにおける構造的性質が Recall 0.0%として顕在化していたことを，正解データ付きで定量的に示した．

(2) 全関数の P-Code 命令列から間接呼び出しを直接列挙し，呼び出し先アドレスの算出に用いられる定数オフセットを JNI 関数テーブルと照合して関数名を復元する手法を提案した．

(3) Recall に基づく定量評価により，C 言語単体で 0.0%から 97.0%，全体で 41.7%から 95.6%への向上を示し，さらに実アプリ 200 件への適用により実環境での有効性と限界を明らかにした．

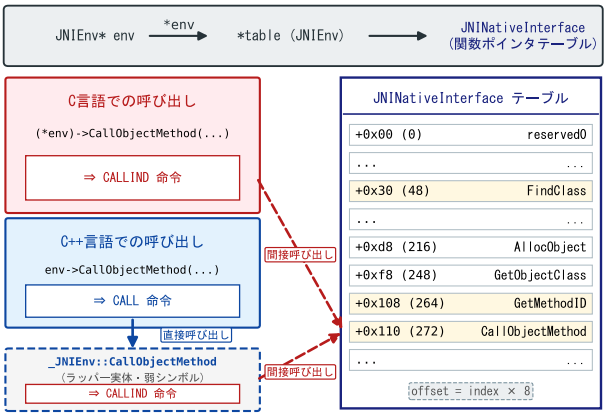


図 1 JNIEnv の二重ポインタ構造と C/C++言語における呼び出し形態の対比

Fig. 1 Comparison of JNIEnv double-pointer structure and call forms in C/C++ languages

本稿の構成は以下の通りである．2 章では本研究の背景と先行研究について述べ，3 章で JNI 呼び出し解析における課題を明確にする．4 章では提案手法の設計と実装を説明し，5 章で検証用データセットの設計，6 章で評価方法について述べる．7 章および 8 章ではそれぞれ検証用データセットと実アプリケーションによる評価結果を示し，9 章で提案手法の有効性と限界を考察する．最後に 10 章で本稿をまとめ，今後の課題を述べる．

2. 背景と先行研究

2.1 JNI とネイティブライブラリの構造

JNI は，Java 仮想マシンと C 言語や C++で記述されたネイティブコードとの相互運用を可能にする規格である [3]．ネイティブコードから Java 層のメソッドを呼び出す場合，以下の 3 ステップの手順を踏む．

(1) `FindClass` 関数により，呼び出し対象の Java クラスの識別情報である `jclass` を取得する．

(2) `GetMethodID` 関数により，クラス情報，メソッド名文字列，シグネチャ文字列からメソッド識別情報である `jmethodID` を取得する．

(3) `Call*Method` 関数などの JNI 関数を実行し，引数を渡して実際に Java メソッドを呼び出す．

ここで重要なのは，`JNIEnv` の実体が関数ポインタテーブルへのポインタである点である．図 1 に JNIEnv の構造と C 言語・C++における呼び出し形態の対比を示す．JNI はテーブルの先頭アドレスのみをネイティブコードに渡し，個々の JNI 関数へはテーブル先頭からの固定オフセットでアクセスさせる．すなわち，このオフセット値から対応する JNI 関数を一意に特定でき，この性質が 4.2 節で提案する関数名の復元手法の基盤となる．

C 言語では，この構造がそのままソースコードに現れ，`(*env)->CallIntMethod(env, obj, mid, ...)` とい

© 2026 Information Processing Society of Japan

う呼び出しは，必ずテーブルから関数ポインタを読み出して呼ぶ間接呼び出しにコンパイルされる．呼び出し先アドレスは実行時に決まるため，対応するシンボル名は存在しない．一方 C++では，_JNIEnv 構造体がメンバ関数としてラッパーを提供するため，env->CallIntMethod(obj, mid, ...) はラッパー実体への直接呼び出しとして現れ得る．ただし，そのラッパーの内部では C 言語と同じ間接呼び出しが行われており，C++の直接呼び出しは間接呼び出しにシンボル化されたラッパーが一枚被さったものにすぎない．

2.2 リバースエンジニアリングツール Ghidra と中間表現 P-Code

Ghidra[8] は，各種 CPU アーキテクチャのアセンブリ命令を中間表現である P-Code[9] にリフティングして解析するリバースエンジニアリングツールである．P-Code には低レベルな Low P-Code と，アーキテクチャ依存性を排除し型や依存関係が構造化された High P-Code が存在する．High P-Code では，変数やレジスタが Varnode という基本単位で表現され，上位の管理データ構造である HighVariable や HighSymbol を通じて変数の型や名前情報を保持する．また，Ghidra は CUI 環境で自動解析を実行する Headless Analyzer や，スクリプトによる解析処理の拡張機能を備えており，大規模なバイナリの自動解析基盤として適している．

2.3 先行研究：JNI コールバック静的解析ツール P-Tracer

本研究の比較対象となる先行研究は，桜井らが開発した P-Tracer[6] である．動的解析は実行パスのカバレッジに限界があるため，ネイティブライブラリの危険性を網羅的に調査するには静的解析が不可欠である [1] が，バイナリレベルでは JNI コールバックのデータフロー追跡が困難であった．

P-Tracer は，Ghidra の High P-Code を活用し，バイナリレベルでの逆テイント解析によって JNI コールバックから Java 層メソッドのクラス名，メソッド名，シグネチャを特定する静的解析ツールである．その処理は，次の 2 つの要素から構成される．

第一に，解析の起点となる JNI 呼び出し箇所の**列挙**である．解析対象の .so ファイルに対して Headless Analyzer による自動解析を行い，バイナリ内に実体を持つ Call*Method 関数のシンボルを特定し，その XREF を辿ることで，当該 JNI 関数を利用しているネイティブ関数を抽出する．

第二に，列挙した呼び出し箇所を起点とする**逆テイント解析**である．Call*Method 関数を実行する P-Code 命令を根ノードとし，入力オペランドである Varnode の定義元 P-Code を再帰的に遡ることで，第 3 引数の jmethodID から対応する GetMethodID 関数を，さらにその引数からクラス情報を与える FindClass 関数や GetObjectClass 関数，およびメソッド名とシグネチャの文字列定数を抽出し，情報フローグラフとして出力する．リリース用バイナリでは JNI 関数のシンボル名や型情報が削除されるため，JNI 仕様に基づく構造体定義等を保持する JNI 固有の型データベースを用いて，Call*Method 関数の第一引数が JNIEnv*型であるという情報を Ghidra に与えることで，Ghidra は P-Code 上で名前情報を連鎖的に復元可能である．さらに P-Tracer は，JNI_OnLoad 関数やネイティブメソッドを起点に JNIEnv*の伝播を順方向に追跡し，グローバル変数や関数引数を経由する関数間の情報フローにも対応する．

2.4 P-Tracer の限界と本研究の位置づけ

P-Tracer は，P-Code 上での構造化された逆テイント解析により JNI コールバックの全容を静的に把握する基礎技術を確立した．ここで，前節に述べた型情報の再定義処理に着目すると，JNI 呼び出しの解析は次の 2 つの機能に分解できる．

（1）命名　型情報をもとに，名前を失った呼び出しがどの JNI 関数に対応するかを復元する機能．

（2）列挙　バイナリ内のどこが JNI 呼び出しであるかを見つけ出し，解析の起点を与える機能．

図 2 に先行研究の手法と提案手法における呼び出し箇所列挙経路の比較を示す．P-Tracer は（1）を備えていた一方で，（2）を .text 領域に実体を持つ JNI 関数シンボルの XREF のみに依存していた．C 言語の間接呼び出しには辿るべきシンボル実体が存在しないため，型情報によって名前を復元する能力を持っていても，そもそも解析の起点として列挙されない．すなわち，型情報を与えるだけでは間接呼び出しは解析対象にならず，（2）の欠落が検出漏れの根本原因である．本研究は，この（2）を間接呼び出しに対して成立させることで P-Tracer の根本的な課題を解決し，検出網羅性を向上させるものである．

3. JNI 呼び出し解析における課題

各種ビルド条件（5 章）で先行研究の手法の検出能力を詳細に検証した結果，以下の 3 つの課題が特定された．

3.1 課題 A：間接呼び出しの列挙漏れ

先行研究の手法は，JNI 呼び出し箇所を列挙する起点として，バイナリ内に実体を持つ JNI 関数シンボルの XREF を利用していた．しかし，C 言語での JNI 呼び出しはすべて JNIEnv のテーブルを経由する間接呼び出しにコンパイルされ，呼び出し先に対応するシンボル実体が存在しない．このため，XREF を辿る方式では呼び出し箇所が 1 件も抽出されず，Recall は 0.0%となる．

© 2026 Information Processing Society of Japan

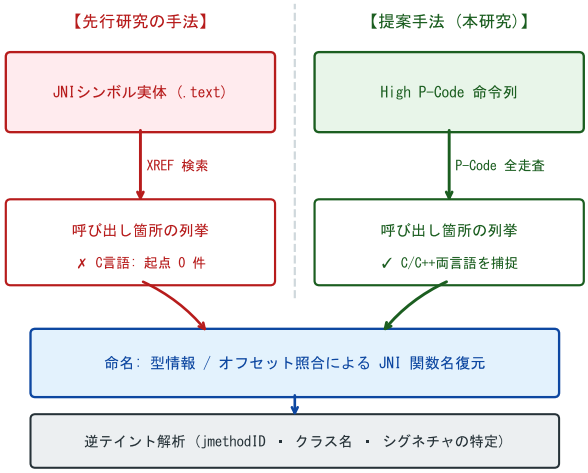


図 2 先行研究の手法と提案手法における JNI 呼び出し箇所の列挙経路の比較

Fig. 2 Comparison of call-site enumeration paths between the prior method and the proposed method

3.2 課題 B：最適化無効時における後方トレースの途絶

最適化および関数のインライン展開が無効化されたビルド条件では，ローカル変数や引数のポインタがスタックフレーム上に頻繁に退避・再ロードされる．先行研究の手法の後方データフロー解析は最適化済みのコード形を前提としていたため，スタック相対参照を経由する多段の LOAD 命令を追跡できず，FindClass 関数に渡される文字列リテラルや JNIEnv ポインタへの到達判定に失敗する．また，間接呼び出しの呼び出し先アドレスを算出するオフセット計算も，最適化無効時にはポインタの部分参照を表す P-Code オペコードである PTRSUB や，値の複製を表す P-Code オペコードである COPY を経由する形で現れるため，定数オフセットに到達できない．

3.3 課題 C：未捕捉例外による解析の中断

Call*MethodV をはじめとする可変長引数を取る JNI 関数を解析する際，引数に対応する Varnode が取得できず None となる場合がある．先行研究の手法では None に対する検査が欠落しており，例外が発生していた．この例外が上位まで伝播することで解析プロセス全体が中断し，当該ライブラリの結果ファイルが生成されずに終了する．外形上は Headless Analyzer が正常終了を報告するため，欠測に気付きにくいという問題もある．

4. 提案手法

上述の課題を解決するため，本稿では以下の手法を設計・実装した．4.1 節および 4.2 節で課題 A に，4.3 節で課題 B に，4.4 節で課題 C について述べる．なお本手法は新しい解析原理を導入するものではなく，2.4 節に述べた（2）列挙を間接呼び出しに対して成立させ，既存の逆テイント解析へ接続するものである．

4.1 P-Code 全走査による間接呼び出しの列挙

課題 A に対して，シンボル依存の列挙による課題を根本的に解決するため，解析対象ライブラリ内のすべての関数を逆コンパイルし，得られた High P-Code の各基本ブロックの命令列を順次走査して，関数ポインタ等を経由した間接呼び出しを表す P-Code オペコードである CALLIND を直接検出する処理を追加した．これにより XREF を一切用いずに解析の起点を得られるため，辿るべきシンボル実体を持たない C 言語の間接呼び出しも捕捉できる．検出した各 CALLIND に対しては，次節の処理により呼び出し先が JNI 関数であるか否かを判定し，JNI 関数であると判定された関数のみを解析対象とする．

4.2 テーブルオフセットの逆引きによる JNI 関数名の復元

検出した間接呼び出しの呼び出し先を同定するため，呼び出し先の JNI 関数名を求める 2 つの探索経路を用いる．

第一の経路は，呼び出し先 Varnode のデータ型名からの復元である．Ghidra が第一引数を JNIEnv*型として復元できている場合，呼び出し先 Varnode のデータ型名が JNI 関数テーブルの要素を表す型名となるため，データ型名と JNI 関数名の対応関係を持つ関数定義の型情報から JNI 関数名を直接得られる．これは 2.4 節の（1）命名にあたり，先行研究の機構をそのまま再利用する．

第二の経路が本稿で開発した処理である．型情報が失われたために第一の経路で名前を得られない場合，呼び出し先 Varnode の定義元 P-Code を逆方向に辿る．この際，型変換を表す P-Code オペコードである CAST およびメモリ読み出しを表す P-Code オペコードである LOAD は読み飛ばし，整数加算を表す P-Code オペコードである INT_ADD に到達した時点でその第二入力の定数を取り出す．得られた値は JNIEnv の指すテーブル先頭からのバイトオフセットであるため，JNI 関数のインデックス順から構築した対応表 [3] と突合することで JNI 関数名を同定する．たとえばオフセット 48 は FindClass 関数に，264 は GetMethodID 関数に，216 は AllocObject 関数に対応する．この方式はシンボルの有無に依存しないため，シンボルが削除されたバイナリに対しても機能する．

名前が判明した間接呼び出しは，直接呼び出しと同一の解析器へ渡され，以降の jmethodID およびクラス情報の逆テイント解析は先行研究の処理をそのまま利用する．

4.3 最適化無効ビルドへの対応

課題 B に対し，オフセットの逆追跡と文字列リテラルの逆解決の双方を拡張した．前者（オフセットの逆追跡の拡張）については，最適化無効時に呼び出し先アドレスの算出が PTRSUB や COPY を経由して現れることに対応するために，逆追跡の走査対象にこれらの命令を追加した．PTRSUB

© 2026 Information Processing Society of Japan

は INT_ADD と同様に第二入力の定数をオフセットとして扱い，COPY は読み飛ばしたために定義元をさらに遡る．

後者については，クラス名の文字列ポインタがスタックスロットを経由して FindClass 関数に渡される形に対応するために，同一関数内の全 P-Code から当該スタックオフセットへ書き込む STORE を探索して格納値を逆解決する処理を追加した．書き込みが複数存在する場合は，条件分岐による合流として複数の候補を保持する．

4.4 解析継続性の確保

課題 C に対し，Varnode の取得処理およびデータフロー解析の各ステップに None に対する事前検査を挿入した．これにより，一部の引数追跡が失敗した場合でも例外による異常終了を発生させず，当該の関数の呼び出し個所であるコールサイトを未解決として扱ったうえで解析を継続し，可能な範囲の結果を出力できる．本改修は解析結果の欠測を防ぐうえで必須であり，4.1 節の列挙処理と併用しなければ，間接呼び出しを多数含むライブラリで結果ファイルそのものが得られない．

5. 検証用データセットの設計

提案手法を検出網羅性の観点から評価するには，各コールサイトがどの Java クラス・メソッドを呼び出すかという正解データが必要である．しかし実アプリではこれを知り得ないため，正解を完全に把握した検証用データセットを構築した．

5.1 実装パターンとビルド構成

本データセットは，JNI コールバックの実装手法を体系的に網羅するため，表 1 に示す A 群から I 群までの観点に基づく 163 種類の実装パターンからなる．同一の Java 処理を多様な実装手法で記述することで，どの実装手法であれば検出でき，どれが困難かを切り分けられる．

同一のソースコードであっても，言語と最適化・インライン展開の設定によって直接呼び出しと間接呼び出しのいずれにコンパイルされるかが変わる．そこでこれらのパターンに対し，C 言語と C++のそれぞれについて最適化無効（インライン展開抑止）と最適化有効の 2 条件，加えて fbjni ラッパーを用いた構成および命名規則による静的登録の構成の計 7 種類のネイティブライブラリをビルドし，呼び出し形態ごとの検出差を測定できるようにした．各構成のバイナリ名を表 2 に，具体的なコンパイル条件を付録 A.3.2 に示す．

5.2 逆アセンブルによる正解データの確立

ツールの自己判定に依存しない中立な正解データを確立するため，Android NDK（Native Development Kit）バージョン 27 の llvm-objdump を用いて全ての.so バイナリを逆アセンブルし，struct JNINativeInterface のメンバインデックスから呼び出し先の JNI 関数名を復元して，実際のコールサイト数を独立に集計した．集計結果を表 2 に示す．全バイナリのコールサイト総数は 897 件であり，その内訳は直接呼び出し 474 件，間接呼び出し 423 件である．この集計結果は，各コールサイトが実際にいずれの呼び出し形態でコンパイルされたかの判定にも用いる．

なお，2.1 節に述べたとおり C++の呼び出しは_JNIEnv 構造体のラッパーを経由するため，NDK の既定設定ではラッパーが弱シンボルの実体として残存し，静的解析上はそこへの直接呼び出し（CALL）として識別される．libptpat_rel_cpp.so で直接呼び出しが最多となるのはこのためである．すなわち呼び出し形態は言語のみで一意に定まらず，最適化やシンボル可視性の設定に依存する．

表 1 実装パターン群の構成
Table 1 Breakdown of implementation pattern groups

群	内容
A	全 10 種類の戻り値の型（Primitive/Object 等）をインスタンスメソッド・静的メソッド・非仮想メソッドの各形式で網羅．jvalue 配列版および可変長引数版（va_list）を含む
B	呼び出し形態の網羅．C++言語のラッパー非展開形式，純粋な C 言語形式，C++言語の完全インライン展開形式，関数ポインタのメモリ退避形式の 4 形態
C	ネイティブメソッドの登録方式．命名規則に基づく静的登録，RegisterNatives 関数による動的登録，および初回の呼び出し時における遅延登録
D	JNIEnv*構造体ポインタの伝播経路．関数の引数での直接受領，構造体メンバへの保存と再ロード，GetEnv 関数経由の取得，ポインタ演算経由の参照
E	jmethodID メソッド識別子のライフサイクル．取得直後の使用，ローカル変数への退避，グローバル変数へのキャッシュ，構造体への保存，引数位置のずれ，条件分岐による合流
F	クラス情報の解決経路．FindClass 関数による文字列リテラル指定，GetObjectClass 関数によるインスタンスからの取得，NewGlobalRef 関数による参照保持
G	ラッパーライブラリ経由の呼び出し．Facebook fbjni 風の中継ライブラリ，SWIG（Simple Wrapper and Interface Generator）風の自動生成コード
H	参照渡しおよびメモリ領域を経由するデータ伝播．ポインタ経由の jmethodID の受け渡し，構造体配列内のメンバの追跡
I	解析スケールと堅牢性の検証．同一メソッドの 150 回を超える連続呼び出し，およびシンボル・型メタデータ欠落時における頑健性

6. 評価方法

6.1 判定基準と評価単位

各評価対象に対する解析結果を，以下の 3 段階に分類する．

© 2026 Information Processing Society of Japan

表 2　逆アセンブルによるコールサイト数の独立集計
Table 2　Independent aggregation of call-site counts via disassembly

バイナリ	直接呼び出し	間接呼び出し	合計
libptpat_dbg_c.so	0	208	208
libptpat_rel_c.so	0	209	209
libptpat_dbg_cpp.so	208	1	209
libptpat_rel_cpp.so	263	4	267
libptpat_fbjni.so	2	0	2
libptpat_static.so	0	1	1
libptpat_static_cpp.so	1	0	1
合計	**474**	**423**	**897**

- **完全特定**: 対象のクラス名，メソッド名，およびシグネチャのすべてを正しく特定できた状態．
- **クラス未特定**: メソッド名とシグネチャは特定できたが，クラス名の特定に失敗した状態．
- **未検出**: 呼び出し箇所を検出できなかった状態．

評価の単位は，個々のコールサイトではなく**実装パターンとビルド構成の組**とする．本ツールは，同一の JNI 関数が多数呼び出された場合に解析時間の膨大化やループが発生することを防ぐため，各バイナリで 100 回の呼び出しを解析回数の上限としている．そのため，I 群は同一メソッドを 150 回以上呼び出すパターンを含むため，コールサイトの生の件数を条件間で単純に比較できないためである．

163 種類の実装パターンと 7 種類のビルド構成の組のうち，`Call*Method` を含むものは 204 件であり，これを評価対象とする．

6.2　評価指標

本稿は検出網羅性の改善を目的とするため，評価対象 204 件のうち当該パターンの JNI コールバックを検出できたものの割合を再現率である Recall と定義し，主要な評価指標とする．あわせて，クラス名を特定できた割合およびメソッド名を特定できた割合を，特定の詳細度を示す補助指標として用いる．

6.3　評価条件

評価は，先行研究 P-Tracer の実装に相当する先行研究の手法と，本稿の提案手法をすべて有効にした提案手法の 2 条件を対比して行う．解析対象の ABI は arm64-v8a とし，全条件が異常終了およびタイムアウトなく完了した．1 条件あたりの所要時間は 93 秒から 189 秒であった（環境と再現手順は付録 A.4）．

7.　検証用データセットによる評価結果

7.1　全体評価

評価対象 204 件における全体的な検出性能の比較結果を表 3 に示す．先行研究の手法の Recall が 41.7%にとどまるのに対し，提案手法では 95.6%に向上した．メソッド名の特定率も 36.3%から 93.6%へと向上し，呼び出し箇所の検出がメソッド名の特定にはほぼ直結した一方，クラス名の特定率は 54.9%にとどまる．

7.2　言語別および呼び出し形態別評価

言語別および呼び出し形態別に分解した評価結果を表 4 に示す．呼び出し形態は，5.2 節の逆アセンブルによる実測結果に基づいて分類した．

言語別では，C++の Recall が 82.5%から 94.2%に改善した一方，C 言語では 0.0%から 97.0%へと向上した．先行研究の手法で検出できた 85 件はすべて C++のビルドに由来しており，C 言語のパターンは 101 件中 1 件も検出できていない．全体の Recall の向上の主要因は，C 言語実装への対応である．

呼び出し形態別では，先行研究の手法において直接呼び出しの 84.8%に対し間接呼び出しは厳密に 0.0%であった．提案手法では間接呼び出しが 97.1%に達し，直接呼び出しの 97.5%とほぼ同等となり，呼び出し形態の差が検出網羅性に与える影響はほぼ解消された．なお，直接呼び出しと間接呼び出しが混在するか形態を実測できなかったパターンは，いずれの対象数にも計上していない．

7.3　ビルド構成別の詳細分析

ビルド構成ごとの内訳を表 5 に示す．いずれも対象数は 50 件である．

C 言語の 2 構成は，いずれも先行研究の手法では 0.0%であったものが 96.0%以上に達している．最適化無効の構成が最適化有効の構成を 1 件分下回るのは，課題 B に述べたオフセット計算の形の違いによるものであり，C 言語では最適化の有無が検出網羅性をほとんど左右しない．

一方，最適化を無効化した C++の構成では，メソッド名の特定率が 90.0%に達する一方でクラス名の特定率が 0.0%となっている．これは，最適化無効時の C++において `FindClass` 関数に渡されるクラス名の文字列ポインタがスタックフレーム相対で扱われ，かつラッパー関数を挟むことでデータフロー解析が文字列リテラルに到達できないためである．この構成のみ，4.3 節の拡張によっても改善しなかった．

8.　実アプリケーションを対象とした適用評価

8.1　評価の概要

検証用データセットでの評価に続き，実環境における本手法の実効性を検証するため，先行研究 P-Tracer[6] の評価で用いられた国内 Google Play のアプリセット（2024 年 2 月収集，各 35 カテゴリのランキング上位アプリのうち arm64-v8a 対応 4,716 件）から選定された 200 件の APK を対象として適用実験を実施した．

© 2026 Information Processing Society of Japan

表 3　全体の検出性能の比較（評価対象 204 件）
Table 3　Comparison of overall detection performance (204 evaluation targets)

条件	検出	Recall	完全特定	クラス特定	メソッド特定
先行研究の手法	85	41.7%	36 (17.6%)	36 (17.6%)	74 (36.3%)
提案手法	**195**	**95.6%**	**112 (54.9%)**	**112 (54.9%)**	**191 (93.6%)**

表 4　言語別および呼び出し形態別の Recall の比較
Table 4　Comparison of recall by programming language and call form

区分		対象数	先行研究の手法	提案手法
言語	C++	103	82.5%	94.2%
	C	101	**0.0%**	**97.0%**
呼び出し形態	直接呼び出し	79	84.8%	97.5%
	間接呼び出し	102	**0.0%**	**97.1%**

実アプリでは正解データを得られないため Recall を算出できない．そこで本節では，特定に成功した Java 層メソッドの絶対数，および重複を除いたユニークメソッド数を主指標とする．補助的に，検出したコールサイト数に対するメソッド解決数の割合を実効解決率として併記する．なお，本評価では，JNI と無関係な間接呼び出しに起因する誤検出および追跡を途中で停止したノードを解析処理の内部で除外している．この除外は先行研究の手法と提案手法の双方に同一の基準で適用しており，両条件の比較可能性を損なわない．

8.2　検出性能

比較結果を表 6 に示す．検出コールサイト数は先行研究の手法の 9,133 件から 25,433 件へと 2.78 倍に拡大した．この際，特定に成功したメソッド解決数は 5,476 件から 7,767 件へと 1.42 倍に増加し，重複を除いたユニークメソッド数は 2,660 件から 5,165 件へと 1.94 倍に拡大した．

一方，実効解決率は先行研究の手法の 60.0%から低下している．間接呼び出しを網羅的に列挙した結果，呼び出し先の特定に至らない箇所も同時に増加したためである．ユニークメソッド数が約 2 倍に増えていることから得られる情報量自体は増加しているが，コールサイトあたりの解決効率は低下しており，この点は 9.2 節に述べる課題である．

8.3　C 言語のみで実装されたアプリへの効果

C 言語のみで記述されたネイティブライブラリを持つ APK では，先行研究の手法の検出コールサイト数が 0 件であったのに対し，提案手法では 70 件のコールサイトを検出し 36 件のメソッドを解決した．検証用データセットで観測された C 言語の Recall 0.0%という現象が，実環境でも同様に生じていたと言える．言語構成別の検出内訳は付録 A.5 に示す．

9.　考察

9.1　間接呼び出し解決の有効性

実験結果が示す通り，間接呼び出しの Recall は 0.0%から 97.1%へ改善し，直接呼び出しの 97.5%とほぼ同等の水準に達した．本手法はシンボル名に依存せず，P-Code のオペコードと `JNIEnv` 構造体のオフセットのみを手掛かりとするため，シンボルが削除されたバイナリに対しても機能する．

ここで注意すべきは，先行研究の手法が C++で 82.5%という一見高い Recall を示していた理由である．これは `_JNIEnv::Call*Method` のラッパー実体シンボルが残存し，XREF 列挙の起点として機能していたことによる．すなわち先行研究の手法の検出可否は，言語ではなくシンボル実体の有無に依存していたと言える．実アプリではシンボルの削除や可視性の制限，リンク時最適化によってこの実体が失われ得るため，C++であっても先行研究の手法の性能は容易に低下する．本手法は C 言語への対応にとどまらず，実アプリの C++に対する頑健性を担保するものでもある．

9.2　制限事項と今後の課題

一方で，以下の点に制限が残ることが明らかとなった．

(1) **可変長引数関数における引数解決**: `Call*MethodV` では引数リストが `va_list` として渡されるため，本評価環境である arm64-v8a では P-Code レベルでの引数追跡が途中で途切れ，クラス名やメソッド名を特定できない．

(2) **最適化無効時の C++におけるクラス名復元**: 課題 B で述べたスタックフレーム相対の文字列アドレスとラッパー関数の境界が重なるため，クラス名の特定率が 0.0%にとどまる．ただしこれは最適化を無効化したビルドで確認されたものであり，最適化を有効にしたビルドが大半を占める実アプリで同様に生じるかは今後の確認を要する．

(3) **ラッパーライブラリ経由の呼び出し**: fbjni や SWIG 等のラッパーライブラリを経由する場合，`jmethodID` やクラス情報のポインタが関数境界を越えて抽象化され，定数の後方追跡が打ち切られる．

これらはいずれも，関数境界におけるデータフロー追跡の途絶という共通の構造的要因に起因しており，関数間解

© 2026 Information Processing Society of Japan

表 5　ビルド構成別の検出詳細（各対象数 50 件）
Table 5　Detection details by build configuration (50 targets each)

ビルド構成	先行研究の手法	提案手法		
	Recall	Recall	クラス	メソッド
C++・最適化無効	94.0%	94.0%	0.0%	90.0%
C++・最適化有効	74.0%	98.0%	76.0%	98.0%
C・最適化無効	0.0%	96.0%	70.0%	92.0%
C・最適化有効	0.0%	98.0%	76.0%	98.0%

表 6　実アプリ 200 件における解析性能の比較
Table 6　Comparison of analysis performance on 200 real-world apps

条件	検出	メソッド解決	ユニーク	実効解決率
先行研究の手法	9,133	5,476	2,660	60.0%
提案手法	25,433	**7,767**	**5,165**	30.5%

析の強化が今後の課題である．

また，8.2 節に述べた実効解決率の低下については，間接呼び出しの検出時に呼び出し元のポインタが `JNIEnv*`型であるかを静的に検証する型検査を導入することで，JNI と無関係な間接呼び出しを列挙の段階で排除できると考えられる．検出網羅性と解決効率を両立させるための課題として取り組む．

10. おわりに

本稿では，Android のネイティブライブラリ解析において，従来の静的解析ツールが抱えていた間接呼び出しの検出漏れ問題の根本原因を，JNI 呼び出し箇所の列挙がシンボルの XREF に依存していた点にあると特定し，P-Code 命令列の全走査とテーブルオフセットの逆引きに基づく手法を設計・実装した．163 種類の実装パターンからなる正解データ付き検証用データセットを用いた評価により，Recall を C++で 82.5%から 94.2%へ，全体で 41.7%から 95.6%へ，特に C 言語では 0.0%から 97.0%へ向上させ，呼び出し形態の差による検出網羅性の差をほぼ解消できることを定量的に示した．さらに，実アプリ 200 件への適用により，特定できたユニークメソッド数が 1.94 倍に増加し，C 言語のみで実装されたライブラリを持つ APK では検出が 0 件から可能になることを確認した．

今後は，可変長引数やラッパー関数に対応するための関数間データフロー追跡の実現に加え，間接呼び出しの列挙における `JNIEnv*`型の静的検査の導入に取り組む．

参考文献

[1] Ruggia, A., Possemato, A., Dambra, S., Merlo, A., Aonzo, S. and Balzarotti, D.: The Dark Side of Native Code on Android, ACM Transactions on Privacy and Security, Vol.28, No.2, Article 13, pp.1–33 (2025).

[2] Afonso, V. M., Bianchi, A., Fratantonio, Y., Doupé, A., Polino, M., de Geus, P. L., Kruegel, C. and Vigna, G.: Going Native: Using a Large-Scale Analysis of Android Apps to Create a Practical Native-Code Sandboxing Policy, Proc. 23rd Annual Network and Distributed System Security Symposium (NDSS 2016) (2016).

[3] Oracle Corporation: Java Native Interface Specification, `https://docs.oracle.com/en/java/javase/21/docs/specs/jni/index.html` (accessed 2026-08-16).

[4] Google LLC: JNI Tips, `https://developer.android.com/training/articles/perf-jni` (accessed 2026-08-16).

[5] Wei, F., Lin, X., Ou, X., Chen, T. and Zhang, X.: JN-SAF: Precise and Efficient NDK/JNI-Aware Inter-Language Static Analysis Framework for Security Vetting of Android Applications with Native Code, Proc. 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS 2018), pp.1137–1150 (2018).

[6] 桜井 涼太, 稲吉 弘樹, 掛井 将平, 齋藤 彰一: Android アプリの Native コードにおける P-code を利用した JNI 呼び出しメソッドの識別, コンピュータセキュリティシンポジウム 2024 論文集, pp.425–432 (2024).

[7] Allix, K., Bissyandé, T. F., Klein, J. and Le Traon, Y.: AndroZoo: Collecting Millions of Android Apps for the Research Community, Proc. 13th International Conference on Mining Software Repositories (MSR 2016), pp.468–471 (2016).

[8] National Security Agency: Ghidra Software Reverse Engineering Framework, `https://ghidra-sre.org/` (accessed 2026-08-16).

[9] National Security Agency: P-Code Reference Manual, `https://github.com/NationalSecurityAgency/ghidra/blob/master/GhidraDocs/languages/html/pcoderef.html` (accessed 2026-08-16).

© 2026 Information Processing Society of Japan

付　　録

本付録は，本稿で提案した手法の再現性を確保するための補足情報である．本文の主張の理解に必要な情報はすべて本文に記載しており，本付録はその根拠となる詳細仕様，詳細アルゴリズム，および評価の内訳を補うものである．

A.1　JNI 関数テーブルのオフセット対応

A.1.1　対応表の構築規則

2.1 節に述べたとおり，`JNIEnv` の実体は `struct JNINativeInterface` の関数ポインタテーブルへのポインタであり，各 JNI 関数はテーブル内で固定のインデックスを持つ．4.2 節の第二の経路で用いるオフセットと関数名の対応表は，JNI 仕様 [3] に定義された当該構造体のメンバ順を先頭から数えたインデックス i に対し，式 (A.1) によってバイトオフセット o を与えることで機械的に構築できる．

$$o = i \times s \tag{A.1}$$

ここで s はポインタサイズであり，本稿の評価環境である arm64-v8a では $s = 8$ である．32 ビット ABI を対象とする場合は $s = 4$ として同一の手順で表を構築できる．すなわち本手法は特定の ABI に固有のものではなく，ポインタサイズの指定のみで他の ABI へ移植できる．

表 A·1 に，本稿の逆テイント解析が起点および中継点として参照する主要エントリを示す．`Call⟨Type⟩Method` 系は，戻り値型が Object, Boolean, Byte, Char, Short, Int, Long, Float, Double および Void の順に並び，各型について引数の渡し方が異なる無印，`V`（可変長引数），`A`（`jvalue` 配列）の 3 変種が連続して配置される．したがって当該系は合計 30 エントリの連続領域をなす．`CallNonvirtual⟨Type⟩Method` 系および `CallStatic⟨Type⟩Method` 系も同一の型順・変種順による 30 エントリの連続領域として配置される．

A.1.2　実行ログによる裏付け

図 A·1 に，C 言語で実装したライブラリを提案手法で解析した際の実行ログの抜粋を示す．`CALLIND` 命令の呼び出し先 `Varnode` から定数オフセットが抽出され，表 A·1 との照合によって JNI 関数名が復元される過程が記録されている．抽出されたオフセット 48，264，216 は，表 A·1 の `FindClass` 関数，`GetMethodID` 関数，`AllocObject` 関数と厳密に一致する．最終行は，復元した名前を用いて可変長引数版である `CallIntMethodV` 関数のコールサイトをアドレス `00100d5c` において列挙できたことを示す．

表 A·1　JNI 関数テーブルの主要エントリ（arm64-v8a，$s = 8$）
Table A·1　Key entries in the JNI function table (arm64-v8a, $s = 8$)

JNI 関数（群）	インデックス	オフセット
`GetVersion`	4	32
`FindClass`	6	48
`NewGlobalRef`	21	168
`AllocObject`	27	216
`GetObjectClass`	31	248
`GetMethodID`	33	264
`CallObjectMethod`	34	272
`CallObjectMethodV`	35	280
`CallBooleanMethod`	37	296
`Call⟨Type⟩Method` 系	34–63	272–504

```
CALLIND Pcode: [(register,0x4000,8) CALLIND
(register,0x4040,8), (register,0x4000,8), ...]
[pcode_util] -> Second input is constant.
[pcode_util] -> Added offset: 48
[pcode_util] -> Likely JNI function: FindClass
[pcode_util] -> Added offset: 264
[pcode_util] -> Likely JNI function: GetMethodID
[pcode_util] -> Added offset: 216
[pcode_util] -> Likely JNI function: AllocObject
[detector] -> [CALLIND] CallIntMethodV
at 00100d5c in CallIntMethod
```

図 A·1　オフセット逆引きによる関数名復元の実行ログ（抜粋）
Fig. A·1　Execution log snippet of function name restoration via offset lookup

A.2　詳細アルゴリズム

A.2.1　P-Code 全走査による間接呼び出しの列挙

4.1 節に述べた列挙処理の手順を図 A·2 に示す．解析対象ライブラリ内の全関数を逆コンパイルし，得られた `HighFunction` の各基本ブロックの P-Code 命令列を走査して `CALLIND` オペコードを直接検出する．シンボルおよび XREF を一切参照しない点が，先行研究の列挙処理との本質的な差である．検出した `CALLIND` のうち，図 A·3 の手続きによって復元した名前が解析対象の JNI 関数名の集合に含まれるものだけを起点として採用する．

計算量は解析対象ライブラリ内の全関数の逆コンパイルに支配され，走査自体は全 P-Code 命令数に対して線形である．6.3 節に述べた 1 条件あたり 93 秒から 189 秒という所要時間の大半はこの逆コンパイル処理が占める．

A.2.2　呼び出し先の同定

4.2 節に述べた `ResolveJniName` の手順を図 A·3 に示す．2 行目から 5 行目が型情報を用いる第一の経路（2.4 節の (1) 命名），7 行目以降が本稿で追加した定数オフセットの逆引きによる第二の経路である．

逆追跡の走査対象のうち，`INT_ADD`，`CAST` および `LOAD`

© 2026 Information Processing Society of Japan

```
Input : F  ライブラリ内の全関数集合
Input : T  オフセットと JNI 関数名の対応表
Input : N  解析対象とする JNI 関数名の集合
Output: S  (JNI 関数名, コールサイト) の集合

S <- {}
for each f in F do
  H <- Decompile(f)
  if H = nil then continue
  for each b in BasicBlocks(H) do
    for each p in PcodeOps(b) do
      if Opcode(p) != CALLIND then continue
      n <- ResolveJniName(Input(p,0), T)
      if n != nil and n in N then
        S <- S + {(n, Address(p))}
return S
```

図 A·2 全関数 P-Code 走査による間接呼び出しの列挙

Fig. A·2 Enumeration of indirect calls via full function P-Code scanning

```
Input : v  CALLIND の呼び出し先 Varnode
Input : T  オフセットと JNI 関数名の対応表
Output: JNI 関数名 または nil

// 経路 1: 型情報による命名
hv <- HighVariable(v)
if hv != nil then
  n <- LookupByTypeName(TypeName(hv), T)
  if n != nil then return n
// 経路 2: 定数オフセットの逆引き
p <- Def(v)
while p != nil do
  switch Opcode(p) do
    case INT_ADD, PTRSUB:
      u <- Input(p,1)
      if IsConstant(u) then
        return T[Offset(u)]
      else return nil
    case CAST, COPY:
      p <- Def(Input(p,0))
    case LOAD:
      p <- Def(Input(p,1))
    otherwise: return nil
return nil
```

図 A·3 呼び出し先 Varnode からの JNI 関数名の復元 (ResolveJniName)

Fig. A·3 Restoration of JNI function names from target Varnode (ResolveJniName)

は最適化を有効にしたビルドで現れる形に対応するものであり，PTRSUB および COPY が 4.3 節に述べた最適化無効ビルドへの対応として追加した拡張である．PTRSUB は INT_ADD と同様に第二引数の定数をテーブルオフセットとして扱い，COPY は CAST と同様に通過して定義元をさらに遡る．表 5 に示した C 言語・最適化無効の構成における Recall の 0.0%から 96.0%への改善は，この 2 オペコードの追加による寄与が大きい．

表 A·2 ビルド構成ごとのコンパイル条件

Table A·2 Compilation flags for each build configuration

構成名	最適化	strip	主な呼び出し形態
dbg_c	-O0	なし	間接
rel_c	-O2	あり	間接
dbg_cpp	-O0 -fno-inline	なし	直接（ラッパー残存）
rel_cpp	-O2	あり	直接・間接の混在
fbjni	-O2	あり	ラッパー API 経由
static	—	—	静的登録（C）
static_cpp	—	—	静的登録（C++）

A.2.3 スタックスロット経由の文字列ポインタの逆解決

4.3 節に述べた文字列リテラルの逆解決の詳細を補足する．最適化無効ビルドでは，FindClass 関数に渡されるクラス名の文字列ポインタが，一度スタックスロットへ退避されてから再ロードされる形で現れる．この場合，通常の後方データフロー追跡は退避先のスタックオフセットを表す定数に到達した時点で終端し，文字列リテラルに到達できない．

そこで，追跡がスタック参照で終端した際に，同一関数内の全 P-Code を走査して当該スタックオフセットを書き込み先とする STORE 命令を探索し，その第三入力である格納値に対して後方追跡を再開する．該当する STORE が複数存在する場合は，条件分岐による合流とみなして全候補を保持し，複数の解を持つノードとして出力する．

A.3 検証用データセットの詳細仕様

A.3.1 設計方針

5 章に述べた検証用データセットは，以下の 4 つの方針に基づいて設計した．

(1) 各コールサイトがどのクラス，メソッドおよびシグネチャを呼び出すかを作者が完全に把握し，正解データとして機械可読な形式で記述する．

(2) 1 つの実装パターンが 1 つの検出能力に対応するよう，着目する要因以外の条件を固定する．

(3) 同一のソースコードを複数のビルド構成で出力し，呼び出し形態ごとの検出差を測定できるようにする．

(4) 同一の Java 処理を多様な実装手法で記述し，どの実装手法であれば検出でき，どれが困難かを切り分けられるようにする．

A.3.2 ビルド構成とコンパイル条件

5.1 節に述べた 7 種類のビルド構成の具体的なコンパイル条件を表 A·2 に示す．最適化無効の構成では，インライン展開を明示的に抑止したうえでシンボルを保持している．最適化有効の構成ではシンボルを削除しており，シンボル情報に依存しない解析が要求される条件となる．

© 2026 Information Processing Society of Japan

```
analyzeHeadless <project_dir> <project_name> \
-import <so_dir> \
-postScript main.py <output_dir> \
-scriptPath <script_dir> \
-readOnly -deleteProject \
-analysisTimeoutPerFile 300
```

図 A·4 Headless Analyzer による解析の実行形式
Fig. A·4 Execution command format for Headless Analyzer

A.3.3 誤検出を検出可能にする仕組み

本稿は検出網羅性の改善を目的とするため Recall を主要な評価指標としたが（6.2 節），評価対象のデータセットには解決結果の誤り（誤った解決，misresolution）を検出できる仕組みを組み込んでいる．なお，ここでの誤った解決は解決結果が誤ったクラスやメソッドを指した件数を表し，コールサイト列挙の Precision とは別の指標である．

第一に，どの呼び出しからも実際には呼ばれないデコイメソッドを Java 層に配置している．解析結果がこれらのメソッドを指した場合，それは誤った解決である．第二に，`CallNonvirtual*Method(env, obj, clazz, mid)` を用いた非仮想呼び出しのトラップを配置している．このとき第 2 引数 `obj` の実体型は派生クラスであるが，正解は第 3 引数 `clazz` が指す基底クラスであり，実体型を答えた場合は誤った解決と判定できる．

これらの仕組みにより，評価に用いた全条件においてデコイメソッドおよび基底クラス以外への誤った解決は 1 件も観測されなかった．

A.3.4 正解データの計数手順

5.2 節に述べた正解データの独立集計は，NDK 27 の `llvm-objdump` による逆アセンブル結果に対し，`JNIEnv` のベースレジスタからの定数オフセットを抽出し，付録 A.1 の対応表によって JNI 関数名を復元して計数する．解析ツール自身の出力を一切参照しないため，ツールの検出漏れが正解の分母に影響しない．評価対象 204 件の内訳は，C++が `dbg_cpp` 50 件，`rel_cpp` 50 件，`fbjni` 2 件，`static_cpp` 1 件の計 103 件，C 言語が `dbg_c` 50 件，`rel_c` 50 件，`static` 1 件の計 101 件である．

A.4 評価環境と再現手順

解析対象の ABI は arm64-v8a とし，正解データの独立集計には NDK 27 の `llvm-objdump` を用いた．解析は Ghidra の Headless Analyzer により実行し，1 ファイルあたりの自動解析タイムアウトを 300 秒に設定した．図 A·4 に，単一の `.so` ファイルに対して解析を実行する際のコマンド形式を示す．`.so` ファイルは APK の `lib/arm64-v8a/` から抽出したものを用いる．

先行研究の手法と提案手法の切り替えは，提案手法として追加した各処理を環境変数によって個別に有効化する機構によって行う．先行研究の手法は全ての追加処理を無効化した状態，提案手法は全ての追加処理を有効化した状態に対応する．同一のコードベース上で条件を切り替えるため，両条件の差は本稿で追加した処理のみに限定される．

表 A·3 実アプリ 200 件における言語構成別の検出内訳
Table A·3 Detection breakdown by language composition on 200 real-world apps

言語構成	条件	検出	メソッド解決	ユニーク
C++を含む	先行研究の手法	9,133	5,476	2,660
C++を含む	提案手法	25,363	7,731	5,132
C 言語のみ	先行研究の手法	0	0	0
C 言語のみ	提案手法	70	36	33

A.5 実アプリ評価における言語構成別の内訳

評価に用いた実アプリ 200 件を，C++実装のライブラリを 1 つ以上含むものと，全てのライブラリが C 言語実装であるものに分類した検出内訳を表 A·3 に示す．後者に対して先行研究の手法はコールサイトを 1 件も検出できていない一方，提案手法は 70 件のコールサイトを検出し 36 件のメソッドを解決している．検証用データセットで観測された C 言語の Recall 0.0%が実環境でも生じていたことは，この内訳に現れている．

前者においても，提案手法の検出コールサイト数は 9,133 件から 25,363 件へ，ユニークメソッド数は 2,660 件から 5,132 件へ増加している．9.1 節に述べたとおり，シンボルの削除や可視性の制限，リンク時最適化によって C++の呼び出しも間接呼び出しとして現れ得るため，C++実装を含むアプリにおいても間接呼び出しの解決は有効である．

A.6 ネイティブライブラリ実態調査の詳細

A.6.1 調査手法

1 章に述べた実態調査は，先行研究 P-Tracer[6] で構築されたアプリセット（2024 年 2 月に収集した国内 Google Play のランキング上位 6,357 件，および 2024 年 6 月に AndroZoo[7] 経由で Google Play, AppChina, Anzhi から収集した 16,350 件の計 22,707 件）から無作為にサンプリングした計 12,240 件の APK を対象とする．各 APK について `lib/`以下の `.so` ファイルの有無を確認し，ネイティブライブラリを含む APK に対して代表 ABI（主に arm64-v8a）内の各`.so` をバイトマーカーの走査によって実装言語に分類した．C++標準ライブラリのランタイムなど，アプリ固有の実装を含まないライブラリは集計から除外している．

A.6.2 集計結果

ネイティブライブラリを含む APK は 5,696 件であり，そこから抽出した 114,882 本のライブラリの言語別内訳を表 A·4 に示す．C++と C 言語で全体の 99.9%を占め，Go および Rust による実装は合計 0.1%にとどまる．

© 2026 Information Processing Society of Japan

表 A·4 ライブラリ単位の実装言語の内訳（114,882 本）

Table A·4 Breakdown of implementation languages per library (114,882 libraries)

言語	本数	割合
C++	70,841	61.7%
C	43,932	38.2%
Go	64	0.1%
Rust	45	0.0%
合計	**114,882**	**100.0%**

APK 単位で見ると，ネイティブライブラリを含む APK の 94.9%が C++実装のライブラリを 1 つ以上含み，全てのライブラリが C 言語実装である APK は 5.1%である．すなわち APK 単位では C++が支配的である一方，ライブラリ単位では約 4 割を C 言語が占める．この傾向はデータセットによらず一貫しており，ライブラリ単位の C 言語比率は各データセットで 3 割から 5 割の範囲にある．

以上より，C++のみに対応した解析ではネイティブライブラリの約 4 割を取りこぼすことになり，間接呼び出しの解決は実運用上不可欠であるといえる．

© 2026 Information Processing Society of Japan